NEURAL SPACETIME SEARCH WITH GRTECLYN: A PRACTICAL BASELINE FOR DYNAMICAL METRIC DISCOVERY

NIKITA M. SHIROKOV*
Independent Researcher, Moscow, Russia
(Dated: May 27, 2026)

We outline a practical baseline for turning `GRTeclyn` into a closed-loop search engine for exotic spacetime metrics on a GPU cluster. Building on prior 3D numerical-relativity evolutions of the Ellis–Bronnikov wormhole, we present an automated framework that goes beyond static metric inspection. A Python scheduler proposes parameter vectors representing spatial and extrinsic curvature profiles. We outline two primary pathways to handle the matter sector: directly reconstructing the required matter distribution from the Einstein constraint equations at $t = 0$, or employing an elliptic solver to force the starting geometry to be mathematically consistent with standard, positive-energy matter. The system is designed to run in batches on a GPU cluster, using early-stopping filters to optimize node hours. Promoted candidates are subjected to operational faster-than-light (FTL) tests, a resolution convergence ladder, and physical stability checks to distinguish physical FTL dynamics from coordinate artifacts.

I. INTRODUCTION

Most proposed interstellar metrics are designed by hand. One writes a static, highly symmetric metric ansatz, calculates the resulting stress-energy tensor using the Einstein field equations, and inspects whether it possesses FTL or shortcut properties. This traditional method is biased toward highly simplified, static geometries that humans know how to write down in closed form.

By treating metric discovery as a closed-loop optimization problem, we can search a much wider, non-spherical, and dynamical space of initial data. The previous `GRTeclyn` wormhole study demonstrated that 3D numerical relativity is highly suited for tracking the nonlinear evolution, instability, and collapse of exotic geometries.

Rather than inspecting static equations, the proposed framework couples a Python-based optimization engine with the high-performance C++ GPU-accelerated AMR engine of `GRTeclyn`. This document outlines the physical, mathematical, and computational framework required to run this search engine across a GPU cluster, with a strict emphasis on deriving the matter sector and avoiding coordinate artifacts.

Throughout this draft, "FTL" means effective faster-than-light propagation through a curved spacetime geometry compared with flat-spacetime light travel. It does not imply local superluminal motion relative to the local light cone.

II. GOAL

The objective is to construct an automated numerical-relativity laboratory for exotic spacetime metrics. The pipeline searches for geometries that display four classes of physical behavior:

1. **FTL transport:** a compact passenger region moves effectively faster than a light signal in flat spacetime, while maintaining bounded, survivable tidal forces.
2. **FTL communication:** a physical signal (null ray or scalar pulse) arrives at a detector earlier than it would in a flat-spacetime control run.
3. **Portals:** two distinct mouth-like structures create a stable shortcut.
4. **Wormholes:** a throat-like spatial bridge that survives for multiple light-crossing times without immediately collapsing into a horizon.

To ensure physical viability, the ultimate success criterion is to find stable, non-collapsing FTL geometries that minimize—or entirely eliminate—violations of the standard energy conditions.

A. Optimization as the Engine of Discovery

A foundational question is why a project aimed at the *discovery* of novel, unclassified spacetimes relies so heavily on numerical *optimization*. The answer lies in the dimensionality and physical constraints of the search space.

The configuration space of all possible 3D initial geometries—defined by the spatial metric γ_{ij} and the extrinsic curvature K_{ij}—is infinitely dimensional. If one conducts a blind, unguided search (i.e., random sampling), the probability of finding a viable candidate is vanishingly small. Virtually all randomly generated initial metrics fail immediately: they either violate the Einstein constraints, collapse into immediate singularity horizons, or disperse into trivial flat space.

Optimization provides the steering mechanism necessary to navigate this infinite space. By defining a strict

* shirokov.nm@phystech.edu

objective scoring function—such as maximizing FTL signal arrival time while minimizing tidal forces and penalizing negative energy density—we establish a physics-guided fitness landscape.

The optimizer does not merely "tweak" a pre-existing geometry; instead, it acts as an evolutionary sculptor. Using a flexible basis of radial and angular functions (e.g., spherical harmonics) as its raw material, the optimizer dynamically combines these modes. Through this process, the system can discover highly non-trivial, asymmetric, or rotating geometries—such as multi-mouth portals or stabilized warp-like corridors—that have never been written down in closed-form by hand. In this framework, optimization is not merely parameter tuning; it is the mathematical mechanism of topological and geometric discovery.

III. INITIAL DATA AND MATTER DERIVATION

In general relativity, we cannot prescribe an arbitrary initial geometry without satisfying the Einstein constraint equations at $t = 0$. The initial spatial metric γ_{ij} and the extrinsic curvature K_{ij} are mathematically coupled to the matter's local energy density ρ and momentum density j_i.

We propose two distinct pathways to handle the matter sector within the automated search loop:

A. Path A: Algebraic Matter Reconstruction

In this pathway, the optimizer is permitted to generate any arbitrary spatial geometry. We do not assume a specific matter model. Instead, we use the Einstein equations to algebraically derive the physical "cost" of the proposed geometry.

Before the time evolution starts, `GRTeclyn` evaluates the Hamiltonian and Momentum constraints at every point on the 3D grid. We algebraically reconstruct the required energy density ρ and momentum density j_i at $t = 0$:

$$\rho = \frac{1}{16\pi}\left(R + K^2 - K_{ij}K^{ij}\right), \tag{1}$$

$$j_i = \frac{1}{8\pi} D_j \left(K_i^j - \delta_i^j K\right). \tag{2}$$

Here, R is the 3D Ricci scalar and D_j is the covariant derivative. The Python script reads this derived ρ grid from the diagnostic files. If $\rho < 0$ in any region, the script flags that this geometry requires exotic matter and calculates the integrated volume of negative energy.

B. Path B: Elliptic Constraint Solving

If we wish to strictly forbid exotic matter from the start, we cannot simply guess a geometry and reconstruct ρ. Instead, we fix the matter sector to a standard, positive-energy model (such as a standard scalar field with $\rho \geq 0$).

We then employ an elliptic initial-data constraint solver (such as `GRTresna`, designed to work with the AMReX grid). The optimizer proposes a "target" geometry, and the elliptic solver slightly deforms this target until it mathematically satisfies the constraints while coupled only to our positive-energy matter field. The resulting constraint-satisfying initial data is then passed to `GRTeclyn` for evolution.

IV. CANDIDATE PARAMETERIZATION

To avoid setting up million-point grids directly from the optimizer, the Python script generates a compact parameter vector θ. A decoder turns θ into smooth initial fields on the AMReX grid.

A. Level 1: Radial Recipes

We define the initial fields at $t = 0$ using spherically symmetric radial profiles:

$$q(r) = q_\infty + \sum_n a_n B_n(r), \tag{3}$$

where q represents the CCZ4 fields χ, α, β^i, or K. The coefficients a_n are the parameters sampled by the optimizer, and $B_n(r)$ are smooth radial basis functions (such as Gaussians).

B. Level 2: Angular Recipes

To allow for non-spherical, asymmetric, or rotating geometries, we expand the parameterization using spherical harmonics:

$$q(r, \theta, \varphi) = q_\infty + \sum_{n\ell m} a_{n\ell m} B_n(r) Y_{\ell m}(\theta, \varphi). \tag{4}$$

This enables the search engine to discover rotating warp fields, quadrupolar configurations, and asymmetric wormhole mouths.

C. Level 3: Neural Field Decoder

In the advanced phase, we can parameterize the initial fields using an implicit coordinate-based neural network (such as a SIREN network):

$$(x, y, z) \xrightarrow{\mathrm{NN}(\theta)} \{\chi, \alpha, \beta^i, K, \tilde{A}_{ij}, \phi, \Pi\}. \tag{5}$$

This is the most flexible option but requires a stable training loop and will be implemented only after the Level 1 and Level 2 pipelines are thoroughly validated.

D. Level 4: Hybrid Meta-Optimization via LLM Agents

While Level 1 and Level 2 parameter searches are managed by derivative-free numerical optimizers (such as CMA-ES), we introduce a high-level LLM-based research agent to manage the meta-optimization loop. The LLM agent does not propose raw floating-point coefficients. Instead, it operates on a feedback loop utilizing the diagnostic output of completed batches:

1. **Automated Diagnostics and Debugging:** If a cluster of candidates terminates early with solver failures or constraint explosions, the LLM agent parses the raw compilation and execution logs. It diagnoses whether the failure is physical (e.g., a physical singularity) or numerical (e.g., coordinate shear, boundary reflection, or grid resolution exhaustion) and modifies the CCZ4 gauge parameters or grid settings accordingly.
2. **Dynamic Hypothesis and Basis Selection:** If the numerical optimizer plateaus in a local minimum, the LLM agent generates new physical hypotheses. It instructs the Python scheduler to modify the parameterization, such as enabling specific angular modes, changing the scalar-field potential $V(\phi)$, or altering the initialization of the shift vector β^i.
3. **Automated Code Adaptation:** For complex physical setups, the agent writes example-local C++ headers (e.g., custom potentials or initial-data functors), runs pre-flight compilation checks, and resolves compiler warnings before launching new search episodes.

V. GPU CLUSTER ARCHITECTURE AND PROGRAM BOUNDARY

To efficiently search the parameter space, the pipeline exploits a clean boundary between Python and pre-compiled C++ executable code across a GPU cluster.

A. The Execution Boundary

The division of labor is designed to eliminate compile-time overhead:

- **Python Controller (CPU/Head Node):** Runs the optimization algorithm, prepares a batch of parameter vectors θ, writes them to individual text-based `params.txt` inputs, and dispatches them as parallel tasks to the cluster's Slurm scheduler.
- **C++ Initial Data (GPU/Compute Nodes):** At $t = 0$, the compiled `GRTeclyn` binary starts up on the assigned GPU, reads the coefficients from `params.txt`, and executes a custom initial-data GPU functor to populate the 3D AMReX grid with smooth initial fields.
- **C++ RHS Evolution (GPU/Compute Nodes):** At $t > 0$, the GPU runs the pre-compiled, highly optimized CCZ4 and matter RHS evolution routines (e.g. `CCZ4RHSWithMatter`). Because the general relativity and matter equations are pre-compiled, the search loop runs with zero compilation overhead.

B. Tiered GPU Execution

To prevent a massive search from exhausting cluster node hours, we implement a tiered system:

- **Tier 1 (Broad Exploration):** Each candidate is allocated to a single GPU and run at a low resolution (e.g. $N_{\text{full}} = 64$ with 2 levels of AMR) for a short duration. Early-stopping criteria instantly terminate a run if constraints explode, the lapse collapses everywhere, or the grid becomes unresolved.
- **Tier 2 (Deep Validation):** High-scoring candidates that survive Tier 1 are promoted to run on multi-GPU, multi-node configurations at high resolution ($N_{\text{full}} = 128 \to 256$, with up to 5 levels of AMR) to verify stability and convergence.

VI. SCORING AND ENERGY CONDITIONS

The objective function must reward desirable physical outcomes and heavily penalize non-physical behavior or numerical errors. The total score $S(\theta)$ for a candidate parameter set is defined as:

$$S(\theta) = w_1 F_{\text{FTL}} - w_2 \mathcal{H}_{\text{violation}} - w_3 E_{\text{exotic}} - w_4 T_{\text{tidal}}, \quad (6)$$

where:

- F_{FTL} is the measured FTL shortcut benefit (determined via the operational probe test).
- $\mathcal{H}_{\text{violation}}$ is the integrated volume norm of the Hamiltonian constraint violation. This ensures the optimizer is not rewarded for coordinate tricks that violate Einstein's equations.
- E_{exotic} is the integrated volume of negative energy density ($T_{00} < 0$). By setting the weight w_3 to a

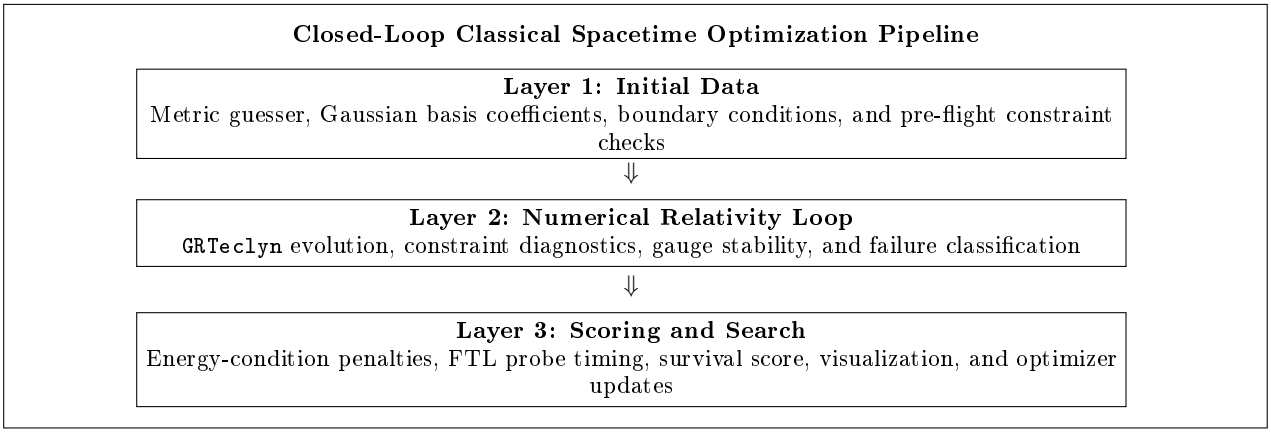

FIG. 1. Schematic diagram of the closed-loop classical spacetime optimization pipeline. Layer 1 handles initial spacetime metric data, boundary conditions, and spatial preprocessing. Layer 2 is the core iterative engine, pairing a numerical relativity solver (such as `GRTeclyn`) with the parameter optimization loop to check physical constraints (including causality and curvature). Layer 3 computes the objective function (evaluating constraint violations, energy conditions, and FTL travel times) and visualizes the resulting 4D embedding structures, directing the overall feedback and metric refinement loop.

very high value, the optimizer is strictly forced to seek configurations supported by normal, positive-energy matter.

- T_{tidal} is the maximum tidal force felt by an observer in the passenger region.

VII. AVOIDING FAKE FTL (OPERATIONAL PROBE TESTS)

A major scientific risk in numerical relativity is "fake FTL," where a coordinate transformation (such as a massive shift vector β^i) makes it look like a region is moving superluminally when it is actually a coordinate effect.

To eliminate coordinate artifacts, every high-scoring candidate must pass a physical operational test:

1. An active probe (either a massless scalar-field pulse or a set of null geodesic tracer particles) is emitted at a boundary A.
2. The pulse propagates through the dynamical spacetime to a detector at boundary B.
3. We compare the arrival time at B with an identical simulation run in flat Minkowski space on the exact same grid.
4. If the signal arrives earlier, we verify that the signal path did not cross an apparent horizon (trapped surface), and that the constraint violations remained bounded during the entire transit time.

VIII. BENCHMARKS

To ensure the pipeline is robust, it must first reproduce known physical configurations prior to starting the open-ended search.

IX. STEP-BY-STEP IMPLEMENTATION PLAN

A. Step 1: Regression and Cleanup

Establish a regression script that reproduces the unperturbed (expanding) and perturbed (collapsing) Ellis–Bronnikov wormhole. Configure the ‘grteclyn-wrapper‘ package to automatically extract the 1D diagnostics and programmatically delete the heavy 3D AMReX plotfiles (‘Plt*‘) to prevent storage exhaustion on the cluster.

B. Step 2: Custom C++ Initial Data

Completed. `Examples/RadialRecipe/` reads Gaussian basis coefficients and optional angular modes from `params.txt` and maps them onto χ, α, β^i, K, ϕ, and Π on the GPU initial-data functor. Short GPU smoke tests confirm the Python→C++ handoff (Sec. XI K).

C. Step 3: Slurm-Based Random Search (The Atlas)

Write the Python scheduler to generate batches of randomized radial coefficients, write `params.txt` inputs, and dispatch them to the GPU cluster via Slurm. Parse the resulting `collapse_diagnostics.dat` files and categorize the failure modes (black hole collapse, coordinate crash, or flat space dissipation) into an `atlas.csv` file.

TABLE I. Initial benchmark suite for the search pipeline.

Case	Expected result	Purpose
Minkowski	Stable, no shortcut	Null/Flat test
Ellis–Bronnikov	Unstable throat	Wormhole detector
Perturbed EB	Collapse + GW signal	Previous-paper regression
Alcubierre-like	High exoticity	Fake-FTL coordinate test
Teo-like	Quadrupole dynamics	Non-spherical benchmark

D. Step 4: Integrate the Constraint Solver

Link `GRTresna` (or an equivalent elliptic constraint solver) into the pre-flight pipeline. Configure the system so that the optimizer's geometric target is deformed to be mathematically consistent with a positive-energy scalar field before starting the C++ time evolution.

E. Step 5: Operational Probe Integration

Implement the active null-ray or scalar pulse propagation test within the `specificPostTimeStep` diagnostic loop of `GRTeclyn` to measure physical arrival times and compute the final FTL score.

F. Step 6: Optimization and Scale-up

Replace the random search with a CMA-ES or Bayesian optimization loop. Once the radial optimization is stable, expand the parameter space to include angular modes (Level 2) and scale the pipeline across multiple GPU nodes.

X. EXPECTED FIRST RESULT

A successful first result is not a claimed working warp drive. The primary deliverable is an automated numerical-relativity classification laboratory.

The first paper will present:

- The verified, open-source closed-loop search pipeline.
- A comprehensive physical database ("The Spacetime Failure Atlas") showing how thousands of random geometries fail dynamically under Einstein's equations.
- The automated discovery of known metric classes (like wormholes) without manual human tuning.
- A rigorous evaluation of the physical viability of FTL metrics under the strict exclusion of negative energy density.

XI. IMPLEMENTED: CONSTRAINT-SATISFYING METRIC GUESSER

The core technical challenge for the automated search is that arbitrary initial data almost always violates the Einstein constraint equations, causing immediate junk radiation, gauge failure, and solver crashes. We have implemented a six-component pipeline that bridges the gap between random metric proposals and physically meaningful evolutions.

A. Physics of the Constraint Gap

For the conformally flat recipe ($\tilde{\gamma}_{ij} = \delta_{ij}$, $A_{ij} = 0$), the Einstein constraints reduce to:

$$R[\chi] + \frac{2}{3}K^2 = 16\pi\rho, \tag{7}$$

$$-\frac{2}{3}D^i K = 8\pi j^i. \tag{8}$$

For a scalar field with $\Pi = 0$ (momentarily static), the momentum density $j^i = -\Pi\,\partial^i\phi = 0$, so the momentum constraint requires $D^i K = 0$—that is, K must be spatially constant. If the recipe generates spatially varying K with $\Pi = 0$, the momentum constraint is violated. If χ and ϕ are independently random, the Hamiltonian constraint is violated.

B. Component 1: Constrained Recipe (Python Pre-Processing)

Rather than guessing ϕ independently, we derive it from χ so that the Hamiltonian constraint is approximately satisfied at $t = 0$. The implementation (`constrained_recipe.py`) performs the following steps:

1. Lock K to a spatial constant (use `recipe_K_asymptotic`, zero out all K Gaussian coefficients). This exactly satisfies the momentum constraint when $\Pi = 0$.
2. Lock $\Pi = 0$ (zero out all Π coefficients).
3. Evaluate $\chi(r)$ on a fine 1D radial grid from the proposed Gaussian basis coefficients.

4. Compute the 3-Ricci scalar using $R = -8\psi^{-5}\nabla^2\psi$ where $\psi = \chi^{-1/4}$.

5. Solve for $\phi(r)$ via radial integration of

$$\left(\frac{d\phi}{dr}\right)^2 = s\,\chi\,\frac{R + \frac{2}{3}K^2}{8\pi}, \tag{9}$$

where $s = +1$ for a normal scalar field and $s = -1$ for a phantom field. Regions where the integrand is negative are flagged as unsatisfiable.

6. Fit the resulting $\phi(r)$ back to Gaussian basis coefficients via least-squares, and inject them into `params.txt`.

This is activated by the `-constrained` flag in the CLI. The optimizer then searches only over χ coefficients and basis parameters; ϕ is always derived consistently.

C. Component 2: Pre-Flight Constraint Filter

Before launching `GRTeclyn` on the GPU, the wrapper evaluates the Hamiltonian and momentum constraint residuals on a coarse 1D radial grid (`preflight.py`). Candidates with $\|H\|_{L^2}$ or $\|M_i\|_{L^2}$ exceeding configurable thresholds, or with $\chi < 0$ in more than 10% of the grid, are rejected without spending GPU time. This is activated by the `-preflight` flag.

D. Component 3: Energy Density Diagnostic (C++)

We extended `RadialRecipeLevel.cpp` to compute the vacuum Hamiltonian constraint (without matter subtraction) at every grid point, giving the *required* energy density:

$$\rho_{\text{required}}(\mathbf{x}) = \frac{1}{16\pi}\left(R + \frac{2}{3}K^2 - A_{ij}A^{ij}\right). \tag{10}$$

Three new columns are written to `constraint_norms.dat` at every diagnostic step: $\min(\rho_{\text{req}})$, $\max(\rho_{\text{req}})$, and the integrated volume of negative ρ_{req}. These allow the Python scorer to evaluate energy-condition compliance without a separate post-processing pass.

E. Component 4: Enhanced Scoring

The episode scorer (`score.py`) now includes two additional weighted components beyond the original survival, constraint health, lapse health, horizon penalty, and nontrivial geometry:

- **Energy condition** (weight 1.5): rewards configurations where $\rho_{\text{req}} \geq 0$ everywhere; penalizes the integrated volume of negative energy density.
- **Initial constraint quality** (weight 1.0): bounded reward based on the initial Hamiltonian L^2 norm, distinguishing clean initial data from noisy starts.

All weights are configurable via a dictionary parameter, enabling task-specific objective functions (e.g., maximizing FTL shortcut benefit vs. minimizing exotic matter).

F. Component 5: CMA-ES Optimization Driver

The random atlas search has been supplemented with a CMA-ES (Covariance Matrix Adaptation Evolution Strategy) optimizer (`optimize.py`). The default search space consists of the four χ Gaussian basis coefficients and the basis width (5 dimensions). Each evaluation runs a full `GRTeclyn` episode and returns the negative total score. The optimizer is invoked via:

```
python -m grteclyn_wrapper \
    --example RadialRecipe \
    --constrained --preflight optimize \
    --max-generations 50 --seed 42
```

CMA-ES automatically adjusts the search distribution based on the fitness landscape, concentrating evaluations in promising regions of the coefficient space.

G. Component 6: Known-Solution Seeds

Three analytically known spacetimes are provided as regression benchmarks and optimizer warm-start points (`seeds.py`):

1. **Flat Minkowski**: $\chi = 1$, all coefficients zero. All constraints are exactly satisfied; the evolution should remain static to machine precision.

2. **Ellis–Bronnikov wormhole**: The exact isotropic χ and ϕ profiles (Eqs. 7 and 11) are fitted to the Gaussian basis via least-squares. This verifies that the recipe basis can faithfully represent a known exotic geometry.

3. **Schwarzschild puncture**: The Brill–Lindquist conformal factor $\psi = 1 + M/(2\bar{r})$ fitted to Gaussians. A vacuum solution that tests the recipe's ability to handle steep χ gradients near a singularity.

These seeds are selected via `-seed-name` and can be combined with the optimizer to search for perturbations around known solutions. Validation-batch IDs (`random_000`, `bubble_wall_016`, ...) are selected via `-candidate-id`; non-spherical proposals via `-nonspherical-id` (see Sec. XI L).

H. Component 7: GPU smoke and promotion scripts

Three bash entry points automate the Python→C++→GPU loop on the cluster:

- `run_radialrecipe_gpu_smoke.sh`: build (optional), dry-run, `check_params`, and short evolution for one seed or candidate;
- `run_radialrecipe_gpu_promote.sh`: resolution ladder at $t = 5$ for promoted radial survivors only;
- `run_nonspherical_gpu_batch.sh`: parallel batch over all ray-sane non-spherical IDs.

Episode outputs live under `runs/radialrecipe_gpu_smoke/`, `runs/radialrecipe_gpu_promote/`, and `runs/radialrecipe_nonspherical/`, each with `params.txt`, `metadata.json`, `run.log`, and `data/constraint_norms.dat`.

I. Verification Tests

All six components are verified by a saved Python test suite:

```
cd grteclyn-wrapper
uv run python tests/test_constrained_guesser.py
```

The script exercises five groups of checks (74 assertions total) before any GPU time is spent.

1. Test groups

1. **Known analytical seeds** (6 cases): flat Minkowski, Ellis–Bronnikov wormholes at $b_0 \in \{0.5, 1.0, 2.0\}$, and Schwarzschild punctures at $M \in \{0.5, 1.0\}$.
2. **Random χ profiles, normal field** (8 cases): χ coefficients drawn from $\mathcal{N}(0, 0.15^2)$ with $N \in [3, 8]$, basis width $w \in [0.3, 2.5]$, and $K \in [-0.3, 0.3]$.
3. **Random χ profiles, phantom field** (8 cases): same layout with larger χ perturbations and $K \in [-0.5, 0.5]$.
4. **Edge cases** (7 cases): very narrow/wide basis, $N = 1$ and $N = 12$, nearly singular χ, non-zero K, depressed asymptotic χ.
5. **Constraint improvement** (5 paired trials): compare pre-flight Hamiltonian L^2 before and after `constrained_overrides`, with random independent ϕ, K, and Π.

2. Results summary

On the reference run (`numpy 2.4.6`, seed 42): **74/74 checks passed**.

- **Robustness**: all 16 random profiles and all 7 edge cases produced finite ϕ coefficients with bounded Gaussian fit residuals (< 1). No NaNs or crashes.
- **Flat Minkowski**: ϕ coefficients remain exactly zero; pre-flight passes with $\|H\|_{L^2} = 0$.
- **Momentum constraint**: locking K to a spatial constant and $\Pi = 0$ drives $\|M_i\|_{L^2} = 0$ in all five improvement trials.
- **Hamiltonian improvement**: in 4/5 random trials, constrained overrides reduced $\|H\|_{L^2}$ (best case: $3.9 \times 10^{-1} \to 2.2 \times 10^{-1}$). One trial regressed slightly ($2.5 \times 10^{-1} \to 2.7 \times 10^{-1}$, $\sim 7\%$) due to Gaussian fit error in the derived ϕ; the test suite allows up to 15% regression to account for this discretisation noise.
- **Pathological rejection**: deliberately bad inputs ($\chi_0 = -5$, spatially varying K) are rejected by pre-flight with multiple independent criteria.

3. Known limitations (expected behaviour)

1. **EB ϕ recovery is approximate, not exact**: re-deriving ϕ from the Gaussian-fitted χ via finite-difference Ricci integration yields $\max|\phi_{\text{orig}} - \phi_{\text{derived}}| \approx 0.2$ for $b_0 = 0.5$. The seed's ϕ was fit analytically; the constraint solver recomputes from the discrete χ profile, so throat-resolution error is expected.
2. **Wide-throat wormholes fail pre-flight**: Ellis–Bronnikov seeds with $b_0 \geq 1.0$ report $\|H\|_{L^2} \sim 10^2$–10^2 on the coarse 1D grid. Steep throat curvature exceeds what 8–12 Gaussians can represent; more basis functions or narrower widths are needed.
3. **Vacuum Schwarzschild fails pre-flight without matter**: puncture χ alone cannot satisfy $H = 0$ with $\phi = 0$ on the recipe's conformally flat ansatz. Pre-flight correctly flags $\|H\|_{L^2} \sim 10^1$–10^2. Full 3D constraint solving (e.g. `GRTresna`) remains required for vacuum BH initial data.
4. **Pre-flight is a heuristic filter, not a proof**: the 1D radial check uses 512–2048 points and finite-difference derivatives. The definitive constraint evaluation happens in the C++ `RadialRecipeLevel` diagnostic loop on the full 3D grid.

These limitations define the intended operating envelope: the constrained recipe is a fast *first pass* that eliminates the worst violations and locks the momentum constraint, while the C++ evolution and scoring pipeline provide the authoritative physics verdict.

J. Guesser Validation Harness

The regression suite in `test_constrained_guesser.py` checks that known cases and edge cases do not crash. A separate batch harness validates the *guesser itself* on synthetic geometries that are **not** exact known metrics, without any optimizer feedback loop.

1. How to run

```
cd grteclyn-wrapper
uv run python tests/validate_metric_guesser.py \
    --output-dir validation_out
# or via CLI:
uv run python -m grteclyn_wrapper validate \
    --output-dir validation_out
```

2. What it does

1. Generates a fixed deterministic candidate set (seed 42): 12 random Gaussian profiles, 3 shells, 3 Alcubierre-inspired bubble walls, 3 multi-shell profiles, 3 pathological cases, and 3 bad controls.
2. Validates each candidate *raw* (random independent ϕ, K, Π) and *constrained* (derived ϕ, locked K, $\Pi = 0$).
3. Records diagnostics: χ range, ϕ fit residual, unsatisfiable-source fraction, required energy density, raw vs. constrained preflight residuals.
4. Classifies each outcome with an explicit label and reason.
5. Writes `guesser_validation.csv` and `guesser_validation_summary.json`.

3. How the initial $\chi(r)$ profile is drawn

The metric guesser starts by proposing the conformal factor $\chi(r)$ on a radial grid. In the C++ recipe this is represented by a Gaussian radial basis:

$$\chi(r) = \chi_\infty + \sum_{n=0}^{N-1} c_n \exp\left[-\frac{(r - r_n)^2}{2w^2}\right], \tag{11}$$

where χ_∞ is the asymptotic value (usually 1), c_n are the guessed coefficients, w is the shared basis width, and the nodes r_n are uniformly spaced from 0 to `recipe_basis_radius_max`. This is the actual shape that `RadialRecipeInitialData` places on the AMReX grid.

The validation harness draws several families of $\chi(r)$:

- **Random Gaussian profiles**: directly sample the coefficients c_n from a controlled normal distribution.
- **Shells**: build a target radial shell, then fit it back into the Gaussian basis.
- **Bubble walls**: use an Alcubierre-style top-hat shape function only as a radial wall proxy, then fit the wall into the Gaussian basis.
- **Multi-shells**: combine two radial shells to test more complex geometries.
- **Pathological/bad controls**: deliberately create steep gradients, narrow basis functions, negative χ, spatially varying K, or random Π.

After $\chi(r)$ is drawn, the constrained recipe does *not* guess $\phi(r)$ independently. Instead, it computes the Ricci scalar from χ, solves the Hamiltonian constraint for $\partial_r\phi$, integrates to obtain $\phi(r)$, and fits that result back into the same Gaussian basis.

4. Current symmetry limitation

The first-stage metric guesser is intentionally spherically symmetric. All guessed fields depend only on the radius,

$$\chi = \chi(r), \qquad K = K(r), \qquad \phi = \phi(r), \qquad \Pi = \Pi(r), \tag{12}$$

with $r = \sqrt{x^2 + y^2 + z^2}$. The spatial metric ansatz is therefore conformally flat and radial,

$$\gamma_{ij} = \chi(r)^{-1}\delta_{ij}. \tag{13}$$

This means the current search can draw smooth spherical blobs, shells, and radial bubble-wall proxies, but it cannot yet represent tilted bubbles, rotating structures, multiple separated mouths, or a true moving Alcubierre bubble.

This restriction is deliberate for the first validation stage: radial symmetry makes it easier to test the constraint solver, energy diagnostics, Gaussian basis fitting, and the Python-to-C++ handoff. Once the present constrained radial solution is validated dynamically, the next symmetry-breaking extension should introduce angular modes, for example spherical harmonics,

$$\chi(r, \theta, \varphi) = \chi_0(r) + \sum_{\ell,m} c_{\ell m}(r) Y_{\ell m}(\theta, \varphi). \tag{14}$$

This would allow lopsided bubbles, multi-mouth portal-like structures, and genuinely non-spherical warp geometries.

5. First non-spherical ray validation

As a first implementation step toward symmetry breaking, the Python wrapper now supports axisymmetric angular deformations of the radial conformal factor:

$$\chi(r,\theta) = \chi_0(r) + \sum_a A_a \exp\left[-\frac{(r-r_a)^2}{2w_a^2}\right] P_{\ell_a}(\cos\theta), \tag{15}$$

where P_ℓ are Legendre polynomials. The same ansatz is now implemented in `RadialRecipeInitialData` (C++) via `recipe_num_chi_angular_modes` and per-mode keys `recipe_chi_mode_ell_*`, `recipe_chi_mode_amp_*`, `recipe_chi_mode_rc_*`, `recipe_chi_mode_rw_*`. A complementary path uses full scalar spherical harmonics $Y_{\ell m}(\theta,\varphi)$ through `recipe_num_chi_Ylm_modes` and `recipe_chi_Ylm_l_*`, `recipe_chi_Ylm_m_*`, etc., evaluated with the existing `SphericalHarmonics::spin_Y_lm` routine ($s = 0$).

The first validation is a one-step metric sanity check along several 1D radial rays ($\theta = 0, \pi/4, \pi/2, 3\pi/4, \pi$). For each candidate we measure:

- the minimum and maximum χ across all sampled rays;
- the fraction of ray samples with $\chi \le 0$;
- the maximum angular contrast $\max_\theta \chi(r,\theta) - \min_\theta \chi(r,\theta)$;
- the corresponding radial stretch range $a(r,\theta) = \chi(r,\theta)^{-1/2}$.

The command is:

```
cd grteclyn-wrapper
uv run python tests/validate_nonspherical_guesser.py \
    --output-dir validation_out
```

The reference run (seed 42) produced 8 non-spherical candidates: 7 were classified as `accepted_ray_sane`, and 1 deliberately strong quadrupole bad-control was rejected as `rejected_negative_chi`. The validation files are:

- `nonspherical_ray_validation.csv`
- `nonspherical_ray_validation_summary.json`

This ray validation only checks whether the non-spherical metric proposal remains positive and geometrically reasonable along sampled directions. It does *not* solve the non-spherical Hamiltonian or momentum constraints on its own. The GPU results in Sec. XIL show that ray-sane candidates also survive short 3D `RadialRecipe` evolution once the constrained ϕ recipe and C++ angular initial data are enabled.

6. Profile visualisation

Representative profiles can be plotted with:

```
cd grteclyn-wrapper
uv run --extra plots python \
    tests/plot_metric_profiles.py \
    --output-dir validation_out
```

This writes one figure per showcase candidate (e.g. `random_000.png`, `bubble_wall_015.png`). Each figure is a 2×2 panel that tells the full pipeline story for a single candidate:

1. **Top-left — Guessed geometry** $a(r) = \chi^{-1/2}$: the radial proper-distance stretch factor. Purple shading above $a = 1$ means space is stretched; orange shading below $a = 1$ means space is compressed. This is the intuitive "shape of space" the guesser proposes.
2. **Top-right — Conformal factor** $\chi(r)$: the internal variable stored on the grid. Red shading below $\chi = 0$ means the metric is invalid.
3. **Bottom-left — Derived scalar field** $\phi(r)$: derived from the Hamiltonian constraint so that matter is consistent with the geometry.
4. **Bottom-right — Required energy density** $\rho_{\rm req}(r)$: *the physics verdict.* Green shading above zero means normal matter can support the geometry. Red shading below zero means exotic (negative) energy is required there.

A bold verdict banner at the bottom of each figure states `ACCEPTED` or `REJECTED` with the classification reason.

7. Representative validation figures

Figures 2, 3, and 4 show the representative validation outputs. Each figure follows the same four-panel format:

1. the guessed radial geometry stretch $a(r) = \chi^{-1/2}$;
2. the stored conformal factor $\chi(r)$;
3. the scalar field $\phi(r)$ derived from the Hamiltonian constraint;
4. the required energy density proxy $\rho_{\rm req}(r)$.

The right-bottom panel is the main physics verdict. Green regions indicate $\rho_{\rm req} \ge 0$ and are compatible with normal positive energy. Red regions indicate $\rho_{\rm req} < 0$, meaning that the geometry requires exotic matter in that radial interval.

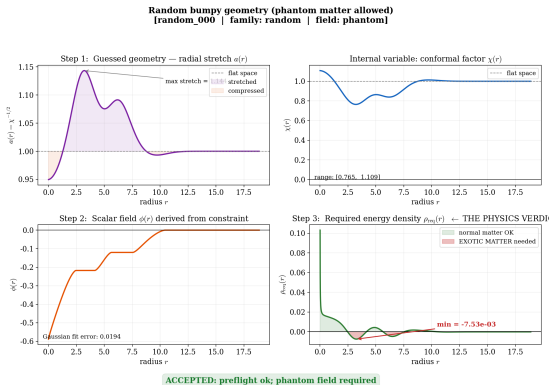

FIG. 2. **Accepted phantom random candidate.** The geometry is numerically constructible and the pre-flight constraints remain controlled. The required energy density contains negative regions, so it is accepted only in the phantom/exotic-matter class.

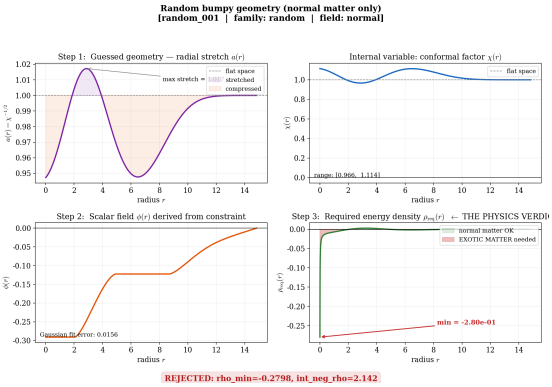

FIG. 3. **Rejected normal-matter random candidate.** The guessed geometry and derived scalar field are well-defined, but $\rho_{\rm req}(r)$ becomes negative. Since this candidate is in the normal-matter class, it is rejected as `rejected_exotic_energy`.

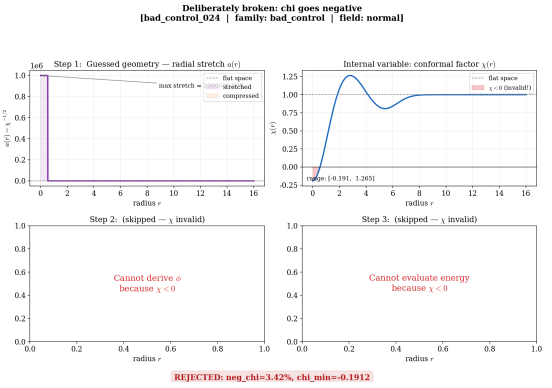

FIG. 4. **Rejected bad-control candidate.** The conformal factor $\chi(r)$ becomes negative, so the spatial metric $\gamma_{ij} = \chi^{-1}\delta_{ij}$ is invalid before any matter reconstruction or evolution.

8. *Representative non-spherical ray figures*

Figures 5 and 6 show the first symmetry-breaking step. These come after the spherical/radial validation figures because the non-spherical ansatz is an extension of the already validated radial pipeline. Each figure should be read as follows:

1. **Top-left**: a physical (x, z) cross-section of $\chi(x, z)$. This is the easiest panel to read as the proposed non-spherical shape.

2. **Top-right**: the corresponding spatial stretch $a(x, z) = \chi^{-1/2}$. Larger values mean more radial proper-distance stretching.

3. **Bottom-left**: the angular shape of χ at the radius where the deformation is strongest. A perfect circle would mean spherical symmetry; departures from a circle show the broken symmetry.

4. **Bottom-right**: the angular contrast $\max_\theta \chi - \min_\theta \chi$ as a function of radius. This shows where the non-spherical deformation is concentrated.

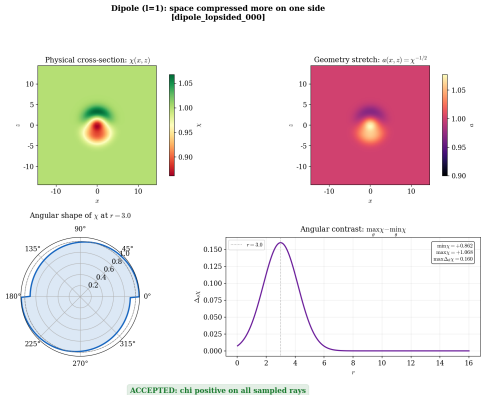

FIG. 5. **Accepted non-spherical dipole candidate.** The $l = 1$ mode makes the geometry lopsided: one side of the cross-section is compressed/stretched differently from the other. The key result is that χ stays positive everywhere in the sampled section, so the metric is geometrically valid at this first ray-check level.

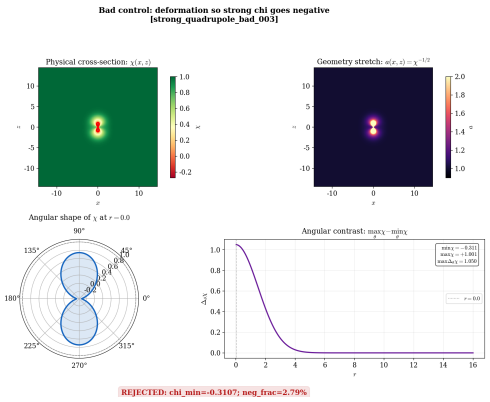

FIG. 6. **Rejected non-spherical bad control.** The same type of angular deformation is made too strong. The cross-section shows regions where $\chi < 0$, which makes $\gamma_{ij} = \chi^{-1}\delta_{ij}$ invalid. This is rejected before any constraint solve or C++ evolution because the metric itself is not admissible.

9. *Classification labels*

- `accepted_normal`: normal scalar supports the geometry; preflight passes; no large negative energy.
- `accepted_phantom`: phantom scalar required; preflight passes (exotic matter allowed).
- `rejected_negative_chi`: invalid metric positivity.
- `rejected_unsatisfiable_source`: Hamiltonian source has wrong sign over most of the grid.
- `rejected_high_constraint`: preflight Hamiltonian or momentum residual too large.
- `rejected_fit_error`: derived ϕ cannot be represented by the Gaussian basis.
- `rejected_exotic_energy`: negative required energy density too large for a normal-matter search.

10. *Reference run (seed 42, 27 candidates)*

- `accepted_phantom`: 7
- `rejected_exotic_energy`: 19
- `rejected_negative_chi`: 1 (bad-control with $\chi_0 = -1.2$)

Most random and shell profiles fail the normal-matter energy check even when preflight passes, because the required energy density ρ_{req} is negative over part of the grid. Phantom-field candidates can pass. This is the intended behaviour: the harness separates *numerical constructibility* from *physical viability under positive energy*.

11. *Difference from regression tests and optimization*

- **Regression tests** (`test_constrained_guesser.py`): fixed assertions on known seeds and edge cases; pass/fail for CI.
- **Validation harness** (`validate_metric_guesser.py`): dataset-style batch over synthetic proposals; explains *why* each geometry is accepted or rejected.
- **Optimization** (`optimize` subcommand): feedback loop that adapts coefficients from scores. Not used here.
- **GPU bridge (completed May 2026)**: short `RadialRecipe` C++ evolutions now run from the wrapper for seeds, validation candidates, and non-spherical proposals. See Secs. XIK–XIL.

K. GPU/C++ RadialRecipe smoke validation

The Python wrapper and `Examples/RadialRecipe/` executable are now connected end-to-end on a single NVIDIA H100 GPU. The automated pipeline is implemented in `grteclyn-wrapper/scripts/run_radialrecipe_gpu_smoke.sh`. It invokes the wrapper's `reproduce` subcommand with `-constrained`, `-preflight`, and `-phantom` as needed.

1. *Build and launch conventions*

Following the same pattern as `SupportedWormholeCollapse`, single-GPU smoke builds use

```
cd Examples/RadialRecipe
make COMP=gnu USE_CUDA=TRUE USE_MPI=FALSE \
    CUDA_ARCH=90 -j
```

This produces `main3d.gnu.CUDA.ex` without requiring `mpicxx` on `PATH`. Multi-GPU production runs remain available via `USE_MPI=TRUE` and the local OpenMPI install under `simulation/local/openmpi-5.0.8`. The wrapper selects `main3d.gnu.MPI.CUDA.ex` only when `-mpi-ranks`> 1.

Each episode performs: (i) Python dry-run generation of `params.txt`; (ii) C++ `check_params=1`; (iii) short GPU evolution; (iv) parsing of `constraint_norms.dat`, `collapse_diagnostics.dat`, and `score.json`.

2. *Initial-data sources*

The CLI now accepts three equivalent entry points, resolved in `candidates.py`:

- `-seed-name` (e.g. `ellis_bronnikov`) from `seeds.py`;
- `-candidate-id` (e.g. `bubble_wall_016`, `random_000`) from the deterministic validation set (`validate_guesser`, seed 42);
- `-nonspherical-id` (e.g. `quadrupole_bubble_001`) from `nonspherical_guesser.py`, mapped to C++ angular-mode parameters by `nonspherical_to_overrides()`.

For every constrained run the wrapper zeros all `recipe_K_coeff_*` and `recipe_Pi_coeff_*`, derives `recipe_phi_coeff_*` from χ, writes the episode `params.txt`, and launches the CUDA binary with `CUDA_VISIBLE_DEVICES` set per job.

3. *Radial smoke results ($t = 2$, $N_{\rm full} = 64$, $\Delta t = 0.02$)*

Table II summarizes the first GPU handoff campaign on the reference phantom candidates. The rough acceptance target is $\|H\|_{L^2} < 10^{-1}$ with no immediate lapse collapse.

The Ellis–Bronnikov seed illustrates the expected mismatch between 1D pre-flight and 3D AMReX: the coarse radial fit can report a large 1D residual even when the 3D diagnostic remains $\mathcal{O}(10^{-2})$. For `bubble_wall_016`, the 3D residual is comparable to or below the Python estimate.

Extended evolution to $t = 5$ at $N_{\rm full} = 64$ confirms `ellis_bronnikov` and `bubble_wall_016` as *promoted survivors* ($\|H\|_{L^2}$ remains $\lesssim 5 \times 10^{-2}$, $\alpha_{\rm min} \gtrsim 0.3$–$0.4$). The same run rejects `random_000` for long campaigns: although it completes without NaNs, $\alpha_{\rm min}$ drops to ~ 0.08 and a large horizon-radius proxy (~ 27 in code units) appears, indicating coordinate/gauge stress. The promotion script `run_radialrecipe_gpu_promote.sh` therefore sweeps only `ellis_bronnikov` and `bubble_wall_016` over $N_{\rm full} \in \{64, 96, 128\}$ at $t = 5$.

4. *Resolution note (Ellis–Bronnikov)*

At $t = 2$, increasing $N_{\rm full}$ from 64 to 128 leaves $\|H\|_{L^2}$ near 5×10^{-2} but exposes sharper throat features ($\chi_{\rm min}$ falls from 0.43 to 0.20, $\alpha_{\rm min}$ from 0.31 to 0.15). Constraints remain controlled; the trend signals under-resolved geometry rather than exponential constraint blow-up.

L. Non-spherical 3D GPU batch

All seven `accepted_ray_sane` non-spherical candidates from the ray-validation set were evolved on GPU in parallel (`run_nonspherical_gpu_batch.sh`, stamp `20260527_161609`, output under `runs/radialrecipe_nonspherical/`). Each run used the axisymmetric C++ deformation of χ, constrained ϕ, $K = 0$, $\Pi = 0$, $t = 2$, and $N_{\rm full} = 64$. The deliberate bad control `strong_quadrupole_bad_003` was not evolved (negative χ on rays).

Every accepted candidate satisfies $\|H\|_{L^2} < 10^{-1}$, maintains $\alpha_{\rm min} \gtrsim 0.87$, and shows no NaN termination. This closes the loop begun in the ray-validation stage: non-spherical χ proposals that are positive along sampled rays also load correctly into C++ and remain dynamically tame over two code units of evolution at low resolution.

Representative commands:

```
# One non-spherical candidate
NONSPHERICAL_ID=quadrupole_bubble_001 BUILD=0 \
  bash grteclyn-wrapper/scripts/\
run_radialrecipe_gpu_smoke.sh

# Full accepted batch (7 jobs, one GPU each)
bash grteclyn-wrapper/scripts/\
run_nonspherical_gpu_batch.sh
```

XII. RECOMMENDED NEXT STEPS FOR TESTING

With the static ($t = 0$) validation harness and the first GPU smoke bridge in place, the remaining short-term protocols below focus on scaling and physics diagnostics rather than basic handoff.

A. C++ 3D Initial Data Discretization Test

Status: completed for reference candidates. Table II records the Python-vs-C++ Hamiltonian comparison for `ellis_bronnikov`, `bubble_wall_016`, and `random_000`. The acceptance target $\|H\|_{L^2} < 10^{-1}$ on the 3D grid is met for the promoted survivors.

- **Remaining work**: systematic sweeps over basis count N and width w to quantify how discretization error scales (see basis fit-residual study below).

B. Basis Fit-Residual Sensitivity Study

The constrained recipe derives a continuous scalar profile $\phi(r)$ and then compresses it back into the finite Gaussian basis used by `RadialRecipe`. If the fit residual is too large, the C++ code will load an inaccurate representation of the matter required by the Hamiltonian constraint, which can cause large violations once the evolution begins.

- **Test**: run the validation harness over a grid of basis counts $N \in [4, 16]$ and widths $w \in [0.3, 2.5]$ for random, shell, and bubble-wall candidate families.

TABLE II. Short GPU smoke results for spherically symmetric RadialRecipe candidates ($t = 2$, $N_{\text{full}} = 64$). Python pre-flight uses a 1D radial grid; C++ values are from `constraint_norms.dat` on the full 3D AMReX mesh.

ID	Python $\|H\|_{L^2}$	C++ max $\|H\|_{L^2}$	C++ final $\|H\|_{L^2}$	$\alpha_{\min}$	$\chi_{\min}$	Outcome
`ellis_bronnikov`	2.24	0.050	0.020	0.31	0.43	Stable
`bubble_wall_016`	0.027	0.014	0.006	0.94	0.99	Stable
`random_000`	0.224	0.092	0.079	0.26	0.77	Borderline

TABLE III. Non-spherical GPU smoke results ($t = 2$, $N_{\text{full}} = 64$, constrained phantom recipe).

Candidate	max $\|H\|_{L^2}$	final $\|H\|_{L^2}$	$\alpha_{\min}$	$\chi_{\min}$
`dipole_lopsided_000`	0.0118	0.0071	0.90	0.88
`quadrupole_bubble_001`	0.0125	0.0074	0.92	0.87
`mixed_modes_002`	0.0118	0.0071	0.91	0.87
`random_angular_004`	0.0064	0.0043	0.92	0.87
`random_angular_005`	0.0122	0.0081	0.92	0.87
`random_angular_006`	0.0132	0.0073	0.91	0.87
`random_angular_007`	0.0135	0.0076	0.93	0.87

- **Measure**: map N and w against the scalar fit residual ϵ_{fit} and the resulting pre-flight Hamiltonian residual.

- **Acceptance target**: determine the smallest basis size that keeps $\epsilon_{\text{fit}} < 10^{-3}$ for non-trivial geometries.

This study will set the minimum safe dimensionality for CMA-ES: small enough to optimize efficiently, but large enough to preserve constraint accuracy.

C. Dynamic Gauge Stability Test

Status: partially completed. `ellis_bronnikov` and `bubble_wall_016` survive $t = 5$ at $N_{\text{full}} = 64$ with controlled constraints and healthy lapse. `random_000` fails this gate at $t = 5$ ($\alpha_{\min} \sim 0.08$, large horizon proxy). All seven ray-sane non-spherical candidates pass the $t = 2$ stability check (Table III).

- **Remaining work**: extend non-spherical survivors to $t = 5$ and the $N_{\text{full}} = 64, 96, 128$ promotion ladder; add FTL probe and Weyl extraction scoring once baselines are frozen.

XIII. LONG-TERM PRODUCTION ROADMAP

Once the short dynamic tests are verified, the pipeline can scale from single-candidate validation to production search.

A. Integrate Multi-Observer Energy Conditions

The current energy diagnostic evaluates the required density in a limited frame. As emphasized by the *Warp Factory* approach, checking only one observer can miss violations seen by boosted timelike or null observers. The production diagnostic should evaluate the stress-energy tensor against a family of local observer vectors.

$$\Xi_W(\mathbf{x}) = T_{\hat{\mu}\hat{\nu}} V^{\hat{\mu}} V^{\hat{\nu}}. \tag{16}$$

- **Action**: extend the C++ diagnostics to sample multiple timelike and null vectors $V^{\hat{\mu}}$ at each grid point and record the minimum value of $\Xi_W(\mathbf{x})$ across observer directions.

- **Scoring integration**: feed this multi-observer minimum into `score.py` so that "normal matter" candidates are required to satisfy energy conditions for all sampled observers, not just the Eulerian one.

- **Purpose**: rule out hidden NEC/WEC violations, including cases that look acceptable in one frame but become exotic in another.

B. Run the Slurm-Managed Constraint Atlas

After the C++ discretization and short-evolution tests pass, the next scale-up step is a constrained random atlas.

- **Action**: launch a 1000-candidate random search on the GPU cluster using the new `-constrained` and `-preflight` CLI flags.

- **Goal**: because the pre-flight filter rejects invalid geometries before GPU launch, the resulting `atlas.csv` should be a cleaner database of meaningful failures and survivals.
- **Research value**: this "Spacetime Failure Atlas" can train downstream research agents by showing where physical constructibility boundaries occur in the parameter space.

C. Execute CMA-ES on the Constrained Subspace

The final near-term production step is to replace random sampling with adaptive search.

- **Action**: run a first 50-generation CMA-ES search using the `-constrained` pipeline.
- **Goal**: the optimizer should no longer spend most of its effort rediscovering the constraint equations. The momentum constraint is handled by construction ($K = \text{const}$, $\Pi = 0$), and the Hamiltonian constraint is approximately handled by deriving ϕ from χ.
- **Optimization target**: let CMA-ES focus on shaping the geometry for survival, stability, non-triviality, and eventual FTL probe performance while the pre-flight and energy-condition filters enforce physical sanity.

XIV. CONCLUSION

This project moves the search for interstellar metrics away from static, hand-written equations toward dynamic, automated discovery. By allowing the computer to propose complex, non-spherical initial data, and using the full power of 3D numerical relativity on a GPU cluster to judge the results, we can rigorously map the physical boundaries of superluminal transport and communication.